Project Name: End-to-End MLOps Capstone Project

Alternative Names: MLOps Repo, Capstone Project, Machine Learning Pipeline, AWS Kubernetes Project

GitHub Link: https://github.com/brij26/MLOPS-Capstone-proj

1. Project Purpose & Overview

- **What it does:** This project is a complete, production-grade Machine Learning Operations (MLOps) pipeline. It handles everything from experiment tracking and version control to containerization, CI/CD automation, cloud deployment, and real-time monitoring.
- **Why I built it:** I built this to showcase how to take ML models from local development to a seamless production environment using industry best practices and automated zero-downtime deployments.

2. Tech Stack

- **Backend & API:** Python 3.10+, Flask (REST API).
- **ML & Versioning:** Scikit-learn, MLflow, DagsHub, DVC, Git.
- **Infrastructure & Cloud:** Docker, AWS EKS (Kubernetes), AWS ECR, AWS S3, AWS EC2.
- **CI/CD & Monitoring:** GitHub Actions, Prometheus, Grafana.

3. Architecture Diagram & Design

- **System Flow:** Local development is version-controlled via Git and GitHub, while data and models are versioned using DVC and stored in AWS S3. Experiments are tracked remotely using MLflow and DagsHub. Pushing code triggers a GitHub Actions CI/CD pipeline that tests the code, builds a Docker container, and pushes the image to AWS ECR. Finally, the container is deployed to an AWS EKS (Kubernetes) cluster. A Flask API serves the model endpoints (/predict, /health), while Prometheus scrapes metrics from the /metrics endpoint to visualize request rates and prediction counts in Grafana.

4. Design Tradeoffs

- **Tradeoff 1 - Complexity vs. Scalability (Kubernetes vs. PaaS):** I chose to deploy the Flask API on AWS EKS (Kubernetes) rather than a simpler Platform-as-a-Service like AWS Elastic Beanstalk or App Runner. While EKS introduces steep infrastructure complexity and a higher baseline cost, I made this tradeoff to ensure the system had production-grade orchestration, fault tolerance, and auto-scaling capabilities necessary for handling volatile ML inference workloads.
- **Tradeoff 2 - Custom Observability vs. Managed Services (Prometheus/Grafana vs. CloudWatch):** I opted to set up custom monitoring using Prometheus and Grafana instead of defaulting to AWS CloudWatch. The tradeoff was that I had to manually

configure the scraping jobs (prometheus.yml) and build custom dashboards. However, this provided a cloud-agnostic observability stack and allowed me to track highly specific custom model metrics—like the ratio of 0 vs. 1 predictions—which CloudWatch does not handle easily out of the box.

5. What I Would Do Differently

- **Hindsight adjustments:** If I were to iterate on this pipeline, I would replace the Flask REST API with a dedicated model serving framework like BentoML or KServe. While Flask is excellent for general web APIs, a dedicated serving layer would allow for better inference optimization, such as request batching and GPU utilization. Additionally, I would implement automated data drift detection to automatically trigger the GitHub Actions training pipeline, closing the loop on continuous training.

6. Notable Implementation Details

- **Key Challenge Solved:** Setting up the DVC pipeline (dvc.yaml) to correctly link the data ingestion and preprocessing stages. I had to ensure that the CI/CD pipeline consistently pulled the exact deterministic dataset from the S3 remote before building the Docker image, guaranteeing absolute reproducibility across environments.
- **Testing Framework:** Integrated pytest within the GitHub Actions CI/CD pipeline to automatically execute unit and integration tests on every code push before the Docker image is built.
- **Specific Implementation Detail:** To enable the real-time monitoring dashboard, I explicitly configured the Prometheus scraper to hit the AWS Load Balancer URL on port 5000 at the metrics endpoint exposed by the Flask app. This allows Grafana to visualize not just system health, but the actual real-time count of model predictions being made.